# JAVA COLLECTIONS FRAMEWORK

*Concepts, Interfaces, Implementation Classes, Methods and Examples*

Learning Notes for Object-Oriented Programming

## Learning Outcomes

After studying this document, students should be able to:

- Explain the purpose of the Java Collections Framework and distinguish interfaces from implementation classes.
- Select suitable collection types for ordered data, unique values, and key-value mappings.
- Use common methods to add, access, update, remove, search, and count elements.
- Write Java programs using List, Set, Map, Queue, and iterators.
- Understand the role of generics and the Collections utility class.
- Apply collections to simple application scenarios such as student records, product catalogues, and request processing.

## 1. Introduction to Collections

A collection is an object that groups multiple objects into a single unit. Collections are useful when an application must store, search, update, sort, or process a set of related values.

For example, a university application may keep student names in a list, store unique registration numbers in a set, and associate each student ID with a name using a map.

### 1.1 Why use collections?

- Collections provide standard methods for common operations such as insertion, deletion, searching, and traversal.
- Many collection implementations can grow or shrink dynamically, unlike arrays whose length is fixed after creation.
- Different implementations support different needs, including positional access, uniqueness, sorted data, and key-value lookup.
- Generics provide compile-time type checking and reduce the need for manual type casting.

### 1.2 Arrays and collections

| Feature | Array | Collection |
|---|---|---|
| Size | Fixed after creation | Many implementations can grow or shrink |
| Operations | Basic operations often need to be implemented manually | Provides standard methods for common operations |
| Data structure choice | Array structure | Includes lists, sets, queues and other structures |
| Type usage | Can store primitives or references | Stores objects; wrapper classes represent primitive values |

## 2. The Java Collections Framework

The Java Collections Framework (JCF) is a set of interfaces and classes, primarily in the java.util package, for representing and manipulating groups of objects. Interfaces define the operations available, while implementation classes provide concrete behaviour.

### 2.1 Core interfaces

| Interface | Purpose | Common implementations |
|---|---|---|
| List | Ordered elements; duplicates are allowed; positional access is available | ArrayList, LinkedList |
| Set | Stores unique elements | HashSet, LinkedHashSet, TreeSet |
| Queue | Holds elements waiting to be processed; many queues use FIFO ordering | LinkedList, PriorityQueue |
| Deque | Supports adding/removing elements at both ends | ArrayDeque, LinkedList |
| Map | Stores key-value pairs with unique keys; Map is not a subtype of Collection | HashMap, LinkedHashMap, TreeMap |

### 2.2 Collection versus Collections

Collection is an interface that forms the root of the main collection hierarchy. Collections is a utility class containing static methods that operate on collections, such as sort(), reverse(), shuffle(), min(), and max(). Map belongs to the Collections Framework but does not extend Collection.

### 2.3 Simplified hierarchy

```
Iterable
  └── Collection
      ├── List       → ArrayList, LinkedList
      ├── Set        → HashSet, LinkedHashSet, TreeSet
      └── Queue      → PriorityQueue, Deque
                         └── ArrayDeque, LinkedList

Map (separate interface)
  ├── HashMap
  ├── LinkedHashMap
  └── TreeMap
```

This is a simplified overview of commonly used interfaces and classes, not the complete API hierarchy.

### 3. List Interface

List represents an ordered collection. Elements can be accessed by index, duplicates are permitted, and insertion order is maintained by common List implementations.

### 3.1 Common List methods

| Method | Purpose |
|---|---|
| add(element) | Adds an element to the list |
| get(index) | Returns the element at the specified index |
| set(index, element) | Replaces the element at the specified index |
| remove(index) / remove(object) | Removes an element by index or by matching object |
| size() | Returns the number of elements |
| contains(object) | Checks whether an element is present |
| clear() | Removes all elements |

### 3.2 ArrayList example

ArrayList is a resizable-array implementation of List. It is commonly used when frequent access by index is required.

```
import java.util.ArrayList;
import java.util.List;

public class StudentListDemo {
    public static void main(String[] args) {
        List<String> students = new ArrayList<>();

        students.add("Aman");
        students.add("Priya");
        students.add("Rahul");
        students.add("Aman"); // duplicates are allowed

        System.out.println(students);
        System.out.println("First student: " + students.get(0));

        students.set(1, "Neha");
        students.remove("Rahul");

        System.out.println("Updated list: " + students);
        System.out.println("Count: " + students.size());
```

```
    }
}
```

Key point: List positions are zero-based. The first element is at index 0. Calling get() with an invalid index causes IndexOutOfBoundsException.

### 3.3 LinkedList

LinkedList implements both List and Deque. It can be useful for operations at the beginning or end of the sequence, but index-based access generally requires traversing nodes. The best choice depends on the access and update pattern.

```
import java.util.LinkedList;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> tasks = new LinkedList<>();
        tasks.add("Design");
        tasks.add("Develop");
        tasks.addFirst("Plan");
        tasks.addLast("Test");

        System.out.println(tasks);
        tasks.removeFirst();
        System.out.println("After removing first: " + tasks);
    }
}
```

### 3.4 ArrayList and LinkedList

| Consideration | ArrayList | LinkedList |
|---|---|---|
| Internal structure | Resizable array | Doubly linked nodes |
| Index-based access | Generally fast | Generally slower because traversal is needed |
| Adding/removing at ends | Appending is usually efficient; front/middle changes may shift elements | End operations are efficient |
| Typical use | Frequent reads and index-based access | Deque/list operations where node-based updates fit the workload |

## 4. Set Interface

Set represents a collection that does not contain duplicate elements. It does not provide index-based access. The iteration order depends on the implementation.

### 4.1 Common Set implementations

| Class | Characteristic |
|---|---|
| HashSet | Stores unique elements; does not guarantee iteration order |
| LinkedHashSet | Stores unique elements and maintains insertion order during iteration |
| TreeSet | Stores unique elements in sorted order according to natural ordering or a comparator |

### 4.2 Common Set methods

| Method | Purpose |
|---|---|
| add(element) | Adds an element if it is not already present |
| remove(element) | Removes the specified element |
| contains(element) | Checks whether the element exists |
| size() | Returns the number of unique elements |
| clear() | Removes all elements |

### 4.3 HashSet example

```
import java.util.HashSet;
import java.util.Set;

public class StudentSetDemo {
    public static void main(String[] args) {
        Set<Integer> ids = new HashSet<>();

        ids.add(101);
        ids.add(102);
        ids.add(103);
        ids.add(101); // duplicate is ignored

        System.out.println("IDs: " + ids);
        System.out.println("Unique count: " + ids.size());
        System.out.println("Contains 102? " + ids.contains(102));
    }
}
```

The order printed by HashSet is not guaranteed. Do not rely on its displayed order to represent insertion order.

### 4.4 TreeSet example

```
import java.util.TreeSet;

public class TreeSetDemo {
    public static void main(String[] args) {
        TreeSet<Integer> scores = new TreeSet<>();
        scores.add(72);
        scores.add(95);
        scores.add(81);
        scores.add(72);

        System.out.println(scores); // [72, 81, 95]
    }
}
```

## 5. Map Interface

Map stores entries as key-value pairs. Each key is unique, while values may be duplicated. A Map is useful when a value must be retrieved using a key such as a student ID, product ID, or username.

### 5.1 Common Map implementations

| Class | Characteristic |
|---|---|
| HashMap | Does not guarantee iteration order |
| LinkedHashMap | Maintains insertion order of entries during iteration |
| TreeMap | Orders entries by key using natural ordering or a comparator |

### 5.2 Common Map methods

| Method | Purpose |
|---|---|
| put(key, value) | Adds a mapping or updates the value for an existing key |
| get(key) | Returns the value associated with the key, or null if no mapping exists |
| remove(key) | Removes the mapping for the key |
| containsKey(key) | Checks whether the map contains a key |
| keySet() | Returns a Set view of the keys |
| values() | Returns a Collection view of the values |
| entrySet() | Returns a Set view of key-value entries |

| | |
|---|---|
| size() | Returns the number of mappings |

### 5.3 HashMap example: Product prices

```
import java.util.HashMap;
import java.util.Map;

public class ProductMapDemo {
    public static void main(String[] args) {
        Map<Integer, Double> prices = new HashMap<>();

        prices.put(101, 499.50);
        prices.put(102, 799.00);
        prices.put(103, 299.00);

        System.out.println("Price of 102: " + prices.get(102));

        prices.put(102, 849.00); // updates existing key
        System.out.println("Updated price: " + prices.get(102));

        System.out.println("Contains product 101? "
                + prices.containsKey(101));

        for (Map.Entry<Integer, Double> entry : prices.entrySet()) {
            System.out.println(entry.getKey() + " -> " +
entry.getValue());
        }
    }
}
```

If put() is called with a key that already exists, its associated value is replaced. HashMap permits one null key and null values; TreeMap and other Map implementations may have different restrictions.

### 5.4 List, Set and Map comparison

| Feature | List | Set | Map |
|---|---|---|---|
| Stores | Individual elements | Unique individual elements | Key-value pairs |
| Duplicates | Allowed | Not allowed | Keys unique; values may repeat |
| Index access | Yes, through List methods | No | Access by key |
| Ordering | Maintains list order | Depends on | Depends on |

| | | | |
|---|---|---|---|
| | | implementation | implementation |
| Example use | Student names in a displayed sequence | Unique registration numbers | Student ID mapped to student name |

### 6. Queue and Deque

A Queue holds elements before they are processed. Many queues follow FIFO (First-In, First-Out), where the first element inserted is normally the first element removed. PriorityQueue is an exception to simple insertion-order processing because elements are ordered by priority according to natural ordering or a comparator.

#### 6.1 Queue methods

| Method | Behaviour |
|---|---|
| offer(element) | Attempts to insert an element; returns false if insertion is not possible |
| poll() | Retrieves and removes the head; returns null if empty |
| peek() | Retrieves but does not remove the head; returns null if empty |
| add(element) | Inserts an element; may throw an exception if insertion fails |
| remove() | Removes and returns the head; throws an exception if empty |
| element() | Retrieves the head; throws an exception if empty |

#### 6.2 Queue example: Service requests

```
import java.util.LinkedList;
import java.util.Queue;

public class RequestQueueDemo {
    public static void main(String[] args) {
        Queue<String> requests = new LinkedList<>();

        requests.offer("Request A");
        requests.offer("Request B");
        requests.offer("Request C");

        System.out.println("Next: " + requests.peek());
```

```
        System.out.println("Processed: " + requests.poll());
        System.out.println("Remaining: " + requests);
    }
}
```

Deque (double-ended queue) supports insertion and removal at both ends. ArrayDeque is a commonly used implementation for stack-like or queue-like operations.

### 7. Iterating Through Collections

Iteration means visiting the elements of a collection one by one. Common approaches include the enhanced for loop, Iterator, and forEach().

#### 7.1 Enhanced for loop

```
import java.util.ArrayList;
import java.util.List;

public class IterationDemo {
    public static void main(String[] args) {
        List<Integer> marks = new ArrayList<>();
        marks.add(78);
        marks.add(85);
        marks.add(92);

        for (int mark : marks) {
            System.out.println(mark);
        }
    }
}
```

#### 7.2 Iterator

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorDemo {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("Aman");
        names.add("Priya");
        names.add("Rahul");

        Iterator<String> it = names.iterator();
```

```
        while (it.hasNext()) {
            String name = it.next();
            System.out.println(name);
        }
    }
}
```

An Iterator provides hasNext() to check whether another element is available and next() to retrieve it. When removing elements during iteration, use the iterator's remove() method where supported rather than structurally modifying the collection directly inside an enhanced for loop.

## 8. Sorting and the Collections Utility Class

The java.util.Collections class provides static utility methods for working with collections. These methods are distinct from methods declared by collection interfaces.

### 8.1 Common utility methods

| Method | Purpose |
|---|---|
| Collections.sort(list) | Sorts a list according to natural ordering |
| Collections.reverse(list) | Reverses the order of list elements |
| Collections.shuffle(list) | Randomly rearranges list elements |
| Collections.max(collection) | Returns the maximum element |
| Collections.min(collection) | Returns the minimum element |
| Collections.frequency(collection, value) | Counts occurrences of a value |
| Collections.binarySearch(list, key) | Searches a sorted list and returns the index if found |

### 8.2 Sorting example

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class SortingDemo {
    public static void main(String[] args) {
        List<Integer> marks = new ArrayList<>();
        marks.add(78);
        marks.add(92);
        marks.add(65);
        marks.add(85);

        Collections.sort(marks);
```

```
        System.out.println("Ascending: " + marks);

        Collections.reverse(marks);
        System.out.println("Reversed: " + marks);

        System.out.println("Highest: " + Collections.max(marks));
        System.out.println("Lowest: " + Collections.min(marks));
    }
}
```

binarySearch() should be used on a list sorted according to the same ordering used for the search. If the key is not found, the returned value is negative and can be used to determine an insertion point.

### 9. Generics and Type Safety

Generics allow the element type to be specified when declaring a collection. This helps the compiler detect incompatible values and reduces explicit casting.

```
List<String> names = new ArrayList<>();
names.add("Aman");
// names.add(100); // Compilation error: Integer is not a String

List<Integer> marks = new ArrayList<>();
marks.add(85);
```

Collections store objects, not primitive values directly. Java wrapper classes such as Integer, Double, Character, and Boolean represent primitive values when used with collections. Autoboxing and unboxing convert between primitive values and their corresponding wrapper objects in many contexts.

### 10. Choosing the Right Collection

| Requirement | Possible choice | Reason |
|---|---|---|
| Ordered elements, duplicates allowed, access by index | ArrayList | Maintains sequence and supports positional access |
| Unique elements; order not important | HashSet | Prevents duplicate elements |
| Unique elements in sorted order | TreeSet | Maintains sorted set order |
| Unique elements in insertion order | LinkedHashSet | Retains insertion order during iteration |
| Key-value lookup | HashMap | Retrieves values by unique key |
| Key-value mappings traversed | TreeMap | Orders entries by key |

| | | |
|---|---|---|
| in key order | | |
| Key-value mappings in insertion order | LinkedHashMap | Maintains insertion order |
| Requests processed in queue order | Queue implementation | Supports queue-oriented insertion and retrieval |

### 11. Integrated Example: Student Marks Manager

The following program combines a List with common operations and the Collections utility class.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class StudentMarksManager {
    public static void main(String[] args) {
        List<Integer> marks = new ArrayList<>();

        marks.add(78);
        marks.add(91);
        marks.add(84);
        marks.add(78);

        System.out.println("Original marks: " + marks);
        System.out.println("Number of entries: " + marks.size());

        Collections.sort(marks);
        System.out.println("Sorted marks: " + marks);

        System.out.println("Highest mark: " + Collections.max(marks));
        System.out.println("Lowest mark: " + Collections.min(marks));
        System.out.println("Occurrences of 78: "
                + Collections.frequency(marks, 78));

        marks.remove(Integer.valueOf(78));
        System.out.println("After removing one 78: " + marks);
    }
}
```

Notice that remove(Integer.valueOf(78)) removes the matching object. Calling remove(78) on a List<Integer> would select the index-based remove(int index) overload and may cause an IndexOutOfBoundsException if index 78 does not exist.

## 12. Common Errors and Good Practices

- Do not assume that HashSet or HashMap iterates in insertion order.
- Use List when duplicates or positional access are required; use Set when uniqueness is required.
- Remember that Map is not a subtype of Collection, even though it is part of the Collections Framework.
- Use generics, such as List<String>, instead of raw collection types.
- Check collection size or emptiness before accessing or removing elements where necessary.
- Do not modify a collection structurally while traversing it with an enhanced for loop; use a suitable iterator or another safe approach.
- Use Collections.binarySearch() only when the list is sorted according to the appropriate ordering.
- Choose a collection based on required behaviour rather than assuming one implementation is always fastest.

## 13. Practice Questions

### A. Conceptual questions

1. What is the Java Collections Framework, and why is it useful?
2. Differentiate between Collection and Collections.
3. Compare List, Set, and Map with suitable examples.
4. Why does a Set reject duplicate elements?
5. Differentiate between ArrayList and LinkedList.
6. Compare HashSet, LinkedHashSet, and TreeSet.
7. What is the purpose of generics in collections?
8. What is the difference between poll() and remove() in a queue?
9. Why should binarySearch() be applied to a suitably sorted list?
10. Why is Map considered part of the Collections Framework but not a subtype of Collection?

### B. Programming exercises

11. Create an ArrayList of five course names. Display all names, replace one name, and remove another.
12. Store a series of integer values in a HashSet. Add a duplicate and display the final size.
13. Create a TreeSet of integers and display its values in sorted order.
14. Build a HashMap that maps product IDs to product names. Search for a product by ID and update its name.
15. Use a Queue to simulate three customer requests. Display the next request and process requests in order.
16. Store student marks in a List. Display the minimum, maximum, sorted marks, and frequency of a given mark.
17. Use an Iterator to traverse a list of strings.
18. Create a LinkedHashMap and demonstrate its insertion-order iteration behaviour.

## 14. Summary

The Java Collections Framework offers reusable interfaces and implementations for storing and manipulating groups of objects. List is appropriate for ordered sequences that may contain duplicates, Set is suitable for unique elements, Map represents key-value relationships, and Queue supports elements waiting to be processed. Iterators, generics, and the Collections utility class provide additional ways to traverse, type-check, sort, search, and manipulate data. Selecting the appropriate collection depends on the application's data and operational requirements.